

# What If Prompt Injection Never Left? Exploring Cross-Session Stored Prompt Injection in Agentic Systems

Yuanbo Xie<sup>1,2,\*</sup>, Tianyun Liu<sup>1,2,\*</sup>, Yingjie Zhang<sup>1,2</sup>, Suchen Liu<sup>1,2</sup>, Yulin Li<sup>1,2</sup>,  
Liya Su<sup>3</sup>, Tingwen Liu<sup>1,2,†</sup>

<sup>1</sup>Institute of Information Engineering, Chinese Academy of Sciences, China,

<sup>2</sup>School of Cyber Security, University of Chinese Academy of Sciences, China,

<sup>3</sup>AI Sec Lab, Beijing Chaitin Technology Co.,Ltd

{xieyuanbo,liutianyun,liutingwen}@iie.ac.cn

## Abstract

Modern agentic systems transform LLMs from session-bounded assistants into stateful systems that persist and evolve shared world state across sessions through memories, filesystems, tools, and other long-lived contextual artifacts. This shift fundamentally expands the attack surface of prompt injection. However, prior works on prompt injection have largely focused on model-level threats within a single session, overlooking how cross-session persistent system state fundamentally changes the system-level risk of agentic systems. Inspired by stored cross-site scripting in web systems, we introduce cross-session stored prompt injection, where a successful injection can persist within agentic system state and silently influence future executions long after the original attacker interaction has ended. To systematically study this threat, we formalize stored prompt injection and develop a taxonomy of how adversarial content persists and affects agentic systems across sessions. We further develop a benchmark and sandbox toolkit to evaluate the risks of stored prompt injection, enabling quantitative analysis of attack success across different models, attack goals, and persistence channels. Our findings highlight that persistence transforms prompt injection from an ephemeral model-level threat into a long-lived system-level vulnerability embedded within agent execution state. We hope this work draws broader attention to this emerging threat and motivates the community to systematically study and mitigate system risks arising from persistence in agentic systems<sup>1</sup>.

## 1 Introduction

Large language model(LLM)-powered agentic systems are rapidly evolving from session-bounded assistants into persistent intelligent systems that

maintain and reuse long-lived state across sessions. Modern agents increasingly rely on memories (Park et al., 2023; Packer et al., 2024; Zhong et al., 2024), filesystems, retrieval-augmented generation (Lewis et al., 2020; Guu et al., 2020), tool-use (Zhang et al., 2025a; Patil et al., 2025; Yao et al., 2025; Huang et al., 2025), shared workspaces, and external resources to support personalization, planning, and long-horizon task execution. Unlike traditional stateless LLM-powered chatbots, whose behavior is primarily determined by the immediate interaction context, modern agentic systems increasingly operate over persistent execution environments, where memories, files, tool-visible artifacts, and other long-lived system state may later be dynamically incorporated into the agent’s execution context across sessions.

Existing prompt injection research has largely treated attacks as transient threats confined to a single execution session (Greshake et al., 2023; Pedro et al., 2023; OWASP Foundation, 2023), where malicious instructions are introduced and exploited within the same interaction context. Under this assumption, the attack surface is primarily associated with immediate user inputs or retrieved content directly consumed during generation, while malicious instructions disappear once the session terminates or the context window is cleared. However, persistent agentic systems fundamentally break these assumptions. Once adversarial content can be written into long-lived system state through queries or other input channels, prompt injection is no longer bounded to the lifetime of a single session. Instead, malicious instructions may persist within the agent’s execution environment and later be reincorporated into downstream execution contexts across future sessions, users, and tasks.

This security transition closely resembles the shift from reflected cross-site scripting(XSS) to stored XSS in web systems(Hydara et al., 2015).

\*Equal contribution

†Corresponding Author.

<sup>1</sup>Code: <https://anonymous.4open.science/r/ebb1153e/>

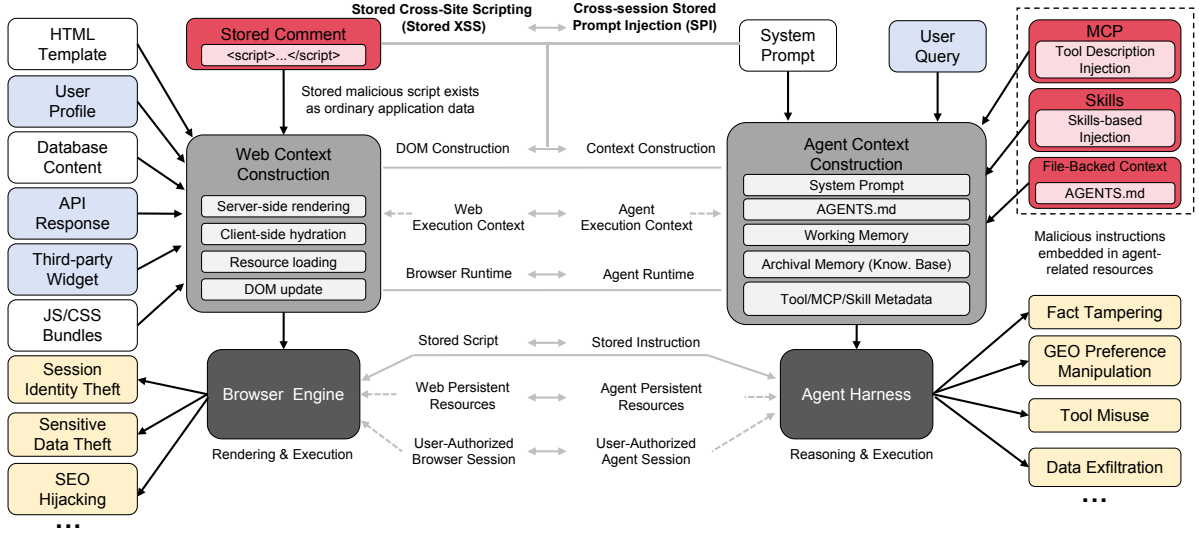


Figure 1: Cross-session Stored Prompt Injection as the Agentic-System Analogue of Stored XSS.

In stored XSS attacks, malicious payloads are first written into persistent application state, such as databases or shared content, and later executed when rendered in downstream user sessions (OWASP Foundation, 2017). The key change is that injection and exploitation become temporally decoupled: attackers no longer need to directly interact with victims at exploitation time. Modern agentic systems are undergoing a similar transition (as shown in Figure 1). Once adversarial instructions are embedded into persistent agent state, such as memories, filesystems, tool-visible artifacts, or shared contextual resources, they may later be reincorporated into future execution contexts across sessions, users, and tasks.

In this paper, we identify and formalize this new class of vulnerabilities, which we term **cross-session stored prompt injection (SPI)** where adversarial instructions written during one interaction persist within agentic system state and later influence downstream executions across future sessions, enabling persistent cross-session attacks. To systematically study this threat, we formalize stored prompt injection and develop a taxonomy for understanding how persistent adversarial context propagates and causes security harms in agentic systems. Based on this taxonomy, we build a benchmark and sandbox toolkit for evaluating SPI attacks across different attack goals, and persistence channels.

Our analysis suggests that SPI attacks fundamentally involve two coupled security problems in persistent agentic systems: unsafe persistent writes and malicious downstream reactivation. During

the write stage, the agent may incorrectly allow attacker-controlled content to be written into long-lived persistent state, particularly strong persistence regions that are later directly incorporated into execution context, creating a toxic cross-boundary data flow from untrusted inputs into trusted system state. During later execution, the runtime context manager may further load contaminated persistent state into downstream execution contexts, allowing adversarial instructions to hijack agent behavior, manipulate responses, or influence tool-mediated actions across sessions. These findings suggest that the security risks of persistent agentic systems arise not only from malicious inputs themselves, but from how agent architectures govern persistent writes and later reincorporation of stored context into execution-time behavior.

Our contributions are summarized as follows:

- We identify a critical security gap emerging from the transition from **stateless conversational assistants** to **stateful agentic systems**, and formalize **cross-session stored prompt injection (SPI)** as a new class of foundational system-level vulnerabilities arising from persistent state.
- Our position paper presents the **first systematic framework and taxonomy** for understanding SPI in agentic systems. By unifying previously fragmented attack surfaces, persistence channels, context incorporation mechanisms, and downstream harms into a coherent model, our framework establishes a common

language for this field, lays the foundation for principled evaluation and mitigation, and opens a new perspective on agent security in the era of agent harness.

- We operationalize the study of SPI through a benchmark and sandbox toolkit covering diverse attack goals and persistence channels. Our findings reveal that exploitability depends on how agent systems govern the full lifecycle of persistent context, from unsafe writes into long-lived state to later incorporation into execution context, highlighting **secure context management as a first-class design principle** for agent harness engineering and a foundational building block for trustworthy next-generation agents.

Ultimately, this work aims to establish cross-session stored prompt injection as a foundational security problem for agentic systems and to motivate broader research on securing persistent context and cross-session agent execution.

## 2 Background and Motivation

Prompt injection normally refers to a class of attacks in which adversarial instructions are embedded within content processed by an LLM and subsequently interpreted as executable directives, causing the model to deviate from its intended objectives, policies, or user instructions (OWASP Foundation, 2023). Rooted in the fundamental difficulty of separating instructions from data in natural language, prompt injection has emerged as one of the most fundamental security threats to LLM-powered systems. As LLM applications have evolved from standalone chat assistants to increasingly agentic systems with access to external resources, tools, and persistent memory, **the threat landscape of prompt injection has evolved accordingly.**

Early prompt injection research focused on *direct prompt injection*, where attacker-controlled user inputs attempt to override trusted system instructions, bypass safety restrictions, or otherwise manipulate model behavior (Perez and Ribeiro, 2022; Liu et al., 2025). The threat model assumes a clear trust boundary between trusted developer-specified instructions and untrusted user inputs, framing prompt injection as an adversarial struggle over instruction authority. Security concerns therefore centered on whether malicious users could subvert the behavioral constraints imposed by system prompts and alignment mechanisms. Under

this formulation, the attacker is the user, the vulnerable asset is the model’s instruction-following behavior, and exploitation occurs within a single interaction session.

As retrieval-augmented generation (RAG) systems (Lewis et al., 2020; Guu et al., 2020) became prevalent, researchers expanded this threat model through the introduction of *indirect prompt injection*, where adversarial instructions are embedded within external resources such as web pages, documents, emails, tool outputs, or retrieved knowledge sources that are later incorporated into an LLM’s context (Greshake et al., 2023). This development fundamentally altered the security assumptions underlying prompt injection. Rather than originating from a malicious user, the attack now arises from untrusted third-party content encountered during the agent’s interaction with its environment. Consequently, the user may be entirely benign while the attacker operates through external information channels that are subsequently consumed by the system. Instead, it concerns the safe integration of external content into the model’s execution context. Prompt injection therefore evolved from a security problem at the user-model interface to one that arises from the integration of untrusted external information into LLM workflows.

In parallel, the emergence of tool-augmented agents (Zhang et al., 2025a; Patil et al., 2025; Yao et al., 2025; Huang et al., 2025) further broadened the prompt injection threat model. Whereas indirect prompt injection primarily concerned the incorporation of untrusted information into model context, modern agents increasingly depend on external tools, MCP services, and reusable skills. These components introduce new trust boundaries, as tool descriptions, capability metadata, execution traces, and service-provided content may all become part of an agent’s reasoning context. Recent work has demonstrated that adversarial instructions embedded within such components can influence planning, manipulate tool-selection decisions, and induce unintended actions (Wang et al., 2026; Guo et al., 2025; Jia et al., 2026). As a result, prompt injection is no longer limited to influencing what an agent knows, but increasingly affects how an agent decides and acts. To study these risks, several benchmarks and evaluation frameworks, including AgentDojo (Debenedetti et al., 2024), Agent Safety Bench (Zhang et al., 2024) and Agent Security Bench (Zhang et al., 2025b), have been proposed to evaluate the security of tool-using agents. Ex-

Existing prompt injection research shares a common underlying assumption: **adversarial content is introduced and exploited within the same execution context**. Whether the attack originates from a malicious user query, a retrieved document, a tool response, an MCP service, or an agent skill, the injected content is observed by the model and influences behavior during the same interaction in which it is introduced. Consequently, existing attacks are fundamentally **session-bound**, relying on immediate context incorporation and immediate exploitation. A recent exception is Memory Injection (Dong et al., 2026), which studies how adversarial content can be planted into an agent’s memory through query-only interactions and later affect behavior. However, its scope remains centered on memory as the persistence target and query-only interaction as the injection channel; it also covers a limited set of downstream harms, leaving broader persistent channels, incorporation mechanisms, attack consequences, and cross-system state contamination largely unexplored. **To the best of our knowledge, no existing benchmark or evaluation framework systematically studies persistent cross-session security risks across diverse injection sources, persistence channels, and context incorporation mechanisms.**

These observations motivate a broader reconsideration of prompt injection in the context of agentic systems. From a system-level perspective, the spatial scope of prompt injection extends beyond individual injection sources to encompass the broader set of system states that may be influenced by adversarial content. Existing research primarily defines prompt injection according to how malicious instructions enter an agent’s context, whether through user inputs, retrieved documents, tool outputs, MCP services, or agent skills. However, in agentic systems, adversarial content does not necessarily remain confined to its original source. Through write operations, it can **propagate** into memories, user profiles, shared workspaces, application state, tool-generated artifacts, and other forms of persistent contextual state. This observation suggests that prompt injection should be understood not only as a problem of adversarial instruction sources, but also as **a problem of state contamination within agentic systems**. Under this view, malicious inputs may exert influence indirectly by poisoning system-visible state, even when they do not immediately alter agent behavior at the time of injection.

Existing prompt injection research also implicitly assumes a strong temporal coupling between injection and exploitation. Whether the attack originates from a malicious user query, a retrieved document, a tool response, an MCP service, or an agent skill, adversarial instructions are typically introduced, incorporated, and activated within the same execution context. Consequently, prompt injection has largely been studied as an immediate and transient threat, where the effects of malicious content are confined to the interaction in which it is first observed. However, this assumption no longer holds in increasingly stateful agentic systems. The growing adoption of long-term memory, shared workspaces, persistent user profiles, MCP-managed resources, and other forms of durable contextual state enables information to outlive the interaction in which it was originally introduced. As a result, adversarial instructions may remain embedded within system state long after the initial injection event and only influence agent behavior when that state is subsequently reincorporated into a future execution context. This observation reveals a fundamentally different class of prompt injection risks in which the injection event and the exploitation event become temporally decoupled. An attacker may introduce malicious content during one interaction, while the resulting behavior is triggered in a different session, by a different user, or under a different task context.

Viewed through this lens, SPI expands the conceptual scope of prompt injection along both spatial and temporal dimensions. Spatially, it shifts attention from individual injection sources to the broader system states that adversarial content can propagate and persist through agent actions. Temporally, it breaks the assumption that injection and exploitation occur within the same interaction, allowing adversarial instructions to persist and activate across sessions. Together, these extensions transform prompt injection from a transient interaction-level threat into a system-level security problem centered on state contamination, persistence, and context construction.

### 3 Preliminaries

#### 3.1 System Model

We model an agentic system as a *context construction pipeline*, in which the input to the language model is assembled from multiple context channels across interaction steps (as shown in Figure 2). At

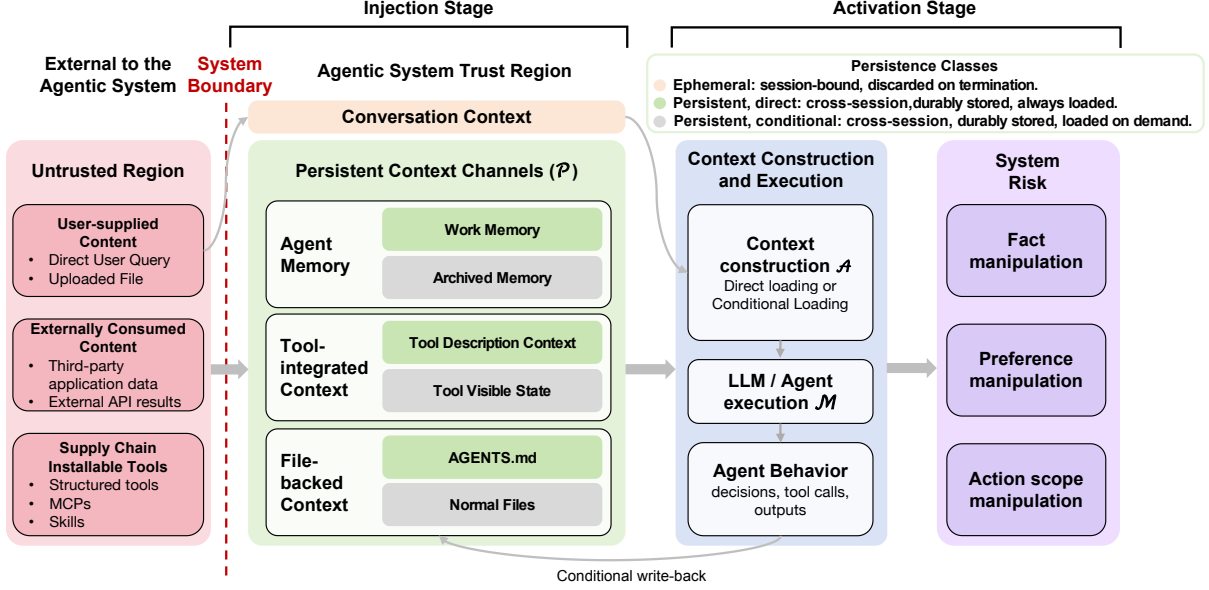


Figure 2: Overview of Stored Prompt Injection Lifecycle.

step  $t$ , given a user query  $q_t$ , the system constructs the agent’s contexts:

$$x_t = \mathcal{A}(q_t, \mathcal{I}, \mathcal{H}_t, \mathcal{T}, \mathcal{R}_t, \mathcal{S}_t), \quad (1)$$

where  $\mathcal{A}$  denotes the context construction function.

The components are defined as follows: (1) Current user query context  $q_t$ : the immediate user input; (2) Instruction context  $\mathcal{I}$ : system- or developer-defined instructions specifying the agent’s role, policies, and behavior; (3) Session history  $\mathcal{H}_t$ : prior interaction context within the current session, including user messages, assistant responses, tool calls, and tool observations; (4) Tool interface context  $\mathcal{T}$ : tool descriptions, function signatures, schemas, MCP metadata, and skill definitions exposed to the model; (5) Retrieved context  $\mathcal{R}_t$ : external knowledge incorporated via retrieval mechanisms, such as documents, vector-store entries, or past records; (6) File-backed context  $\mathcal{S}_t$ : persistent workspace or environment-level data loaded by the agent harness, including files such as `MEMORY.md`, `USER.md`, `SOUL.md`, `AGENTS.md`, or similar artifacts. The assembled input  $x_t$  is processed by the language model  $\mathcal{M}$  to produce output:  $y_t = \mathcal{M}(x_t)$ .

The context channels differ in their lifetime and persistence properties. We first consider *historical context*, which captures information derived from past interactions. Session history  $\mathcal{H}_t$  is typically ephemeral and scoped to a single interaction session. It consists of prior user messages, assistant responses, and execution traces within the current

session, and is usually discarded once the session ends. Retrieved context  $\mathcal{R}_t$ , in contrast, is incorporated from persistent data sources such as memory stores, document collections, or logs of past interactions. Its inclusion in the agent’s context depends on the context construction function  $\mathcal{A}$  and is therefore selective. In many agentic systems, portions of session history may be stored, summarized, or indexed after a session completes, and later accessed through retrieval mechanisms. In such cases, past interaction history is externalized into persistent storage and reintroduced as retrieved context. This introduces a form of conditional persistence, where information persists in the system but is only activated when selected for incorporation into the agent’s context. In contrast, instruction context  $\mathcal{I}$ , tool interface context  $\mathcal{T}$ , and file-backed context  $\mathcal{S}_t$  are typically persistent across sessions and are often incorporated by default or with high likelihood. These constitute strongly persistent context, which may be shared across interactions or users.

Based on their persistence and likelihood of incorporation into model context, we categorize context into three classes: (1) ephemeral context, which is transient and session-bound; (2) conditionally persistent context, which persists but is only incorporated selectively; and (3) strongly persistent context, which is consistently incorporated across interactions. This distinction is critical for understanding how adversarial content propagates and is activated in agentic systems, yet it has been

largely overlooked in prior work. Content introduced in one interaction may persistently influence future agent behavior by modifying persistent context that is incorporated into the agent’s context across later interactions. Therefore, the security boundary lies not only in the model  $\mathcal{M}$ , but in the context construction function  $\mathcal{A}$  and the persistence mechanisms governing these context channels.

**Definition #1 Persistent Context.** Persistent context refers to any information maintained by the agentic system that (i) persists beyond a single interaction and (ii) may be incorporated into the agent’s context through the context construction function  $\mathcal{A}$  in future executions. This definition excludes data that is stored but never incorporated into model context, as such data cannot influence model behavior.

### 3.2 Threat Model

We assume the target agentic system is stateful: external inputs may update memory, retrieval corpora, tool-visible state, or file-backed context, and these persistent context channels may later be incorporated into the agent’s context.

**Adversary’s Capability.** We consider an adversary who interacts with the agentic system through inputs that cross the system boundary from an untrusted region into the agent’s context construction pipeline.

We model this capability as a write primitive from the untrusted region into one or more persistent context channels:  $w : u \mapsto p \in \mathcal{P}$ , where  $u$  denotes adversary-controlled external content and  $\mathcal{P}$  denotes the persistent context reachable by  $\mathcal{A}$ . The write primitive may be realized through different mechanisms, including query-induced memory updates, poisoning of retrieval sources, tool or MCP installation, exposure of malicious external content to browsing or API tools, or modification of file-backed agent state.

**Adversary’s Goal.** The adversary’s goal is not merely to hijack a single model invocation, but to establish a persistent influence over future agent behavior. To this end, the adversary aims to cause malicious content  $p$  to be written into, or to modify, persistent context that is later incorporated into the agent’s context by the context construction function  $\mathcal{A}$ .

**Adversary’s Constraints.** The adversary does not require privileged access to the model  $\mathcal{M}$ , the context construction function  $\mathcal{A}$ , or the victim’s query. In particular, the adversary cannot directly

rewrite the constructed agent’s context, change the agent’s orchestration logic, or force  $\mathcal{A}$  to include arbitrary content at exploitation time. Instead, the adversary can only influence the persistent state that  $\mathcal{A}$  may later load or select during normal agent operation.

**Definition #2 Stored Prompt Injection.** A stored prompt injection attack occurs when adversarial content  $p$  is written into or modifies persistent context  $\mathcal{P}$ , and in a later execution is incorporated into the agent’s context  $x_t = \mathcal{A}(q_t, \cdot, \mathcal{P})$ , where it is interpreted by the model  $\mathcal{M}$  in a way that alters the agent’s behavior beyond the intended task.

This definition separates the injection stage from the activation stage. During injection, the adversary causes malicious content to cross the system boundary and persist within an agent-controlled context channel. During activation, the adversary need not be present; the attack is triggered when normal context construction incorporates the stored content into a future agent’s context. Thus, stored prompt injection exploits persistent state mutation rather than only transient prompt manipulation.

## 4 Taxonomy of Stored Prompt Injection

Stored prompt injection differs from traditional prompt injection in that the adversary aims to establish persistent influence over agent behavior, rather than merely affecting a single agent action. Thus, stored prompt injection is fundamentally a *stateful* attack which targets the agent’s persistent context. Importantly, the impact of such attacks may not manifest within the interaction in which the adversarial content is introduced, but instead emerge in subsequent executions when the stored content is later incorporated. To reason about this class of attacks, it is therefore insufficient to consider only the immediate behavior of the agent within one session. Instead, we must account for **how adversarial content propagates through the agentic system: how it enters, where it persists, and under what conditions it is later incorporated into the agent’s context.**

To answer these questions and capture the attack lifecycle, we organize stored prompt injection along three dimensions: (1) *injection source*, which characterizes how adversarial content crosses the system boundary; (2) *persistent context channel*, which characterizes where such content is stored within the system; and (3) *context incorporation mechanism*, which characterizes how stored con-

tent is later incorporated into the agent’s context. Together, these dimensions characterize the lifecycle of stored prompt injection from entry, to persistence, and finally activation. Together, it provides a structured view of stored prompt injection as a cross-session attack on the persistence mechanism and context construction pipeline of agentic systems.

#### 4.1 Injection Source

The first dimension characterizes where adversarial content first crosses the system boundary and becomes available to the agentic system. These entry points align with prior formulations of prompt injection, encompassing both direct prompt injection (via user-supplied inputs) and indirect prompt injection (via externally consumed content and supply-chain installable tools’ context). However, in the context of stored prompt injection, the role of these sources extends beyond immediate prompt manipulation. Instead, they serve as write channels through which adversarial content may be introduced into the agent’s persistent context. This enables the attacker to establish influence that persists beyond the initial interaction, rather than being confined to a single execution.

**User-Supplied Content.** Adversarial content may be supplied directly through the user-facing interface of the agent, including direct user queries and uploaded files. This corresponds to direct prompt injection, where the attacker explicitly controls the input to the system. In the context of stored prompt injection, such inputs may be written into persistent context (e.g., through memory updates or file modifications), enabling the attacker to influence future executions beyond the initial interaction.

**Externally Consumed Content.** Adversarial content may also be encountered indirectly when the agent consumes external information during task execution, such as third-party application data, web content, or RPC/API results. This corresponds to indirect prompt injection, where the attacker does not directly control the user input but instead influences the content observed by the agent. When such content is ingested and persisted, it may later be incorporated into the agent’s context, allowing adversarial influence to propagate across executions.

**Supply-Chain Installable Tool.** Adversarial content may be embedded in installable tools, such as structured tool definitions, MCP servers, or

skills, which can be introduced into the agent through user-initiated installation. This setting can be viewed as a form of supply-chain-based indirect prompt injection. Adversarial content is carried into the system as part of the tool itself and may subsequently persist within the agent’s operational context.

#### 4.2 Persistent Context Channel

The second dimension captures where injected content persists after entering the system. This dimension is central to stored prompt injection: an attack becomes *stored* when adversarial content is written into, or modifies, context that persists across sessions and can be reintroduced in future interactions. We consider several classes of persistent context channels that are commonly present in agentic systems.

**Agent Memory.** Adversarial content may persist within the agent’s memory system. We distinguish between *working memory*, which is automatically incorporated into the agent’s execution context, and *archival memory*, which is incorporated only when retrieved. Consequently, stored adversarial content may either exert recurrent influence across sessions or remain dormant until reactivated by future retrieval.

**Tool-Visible State.** Adversarial content may persist in tool-integrated context, including tool description context and tool response results. Tool description context is often strongly persistent, as it is loaded as part of the agent’s tool interface (e.g., structured tool definitions or MCP descriptions), though some forms (e.g., skills) may be conditionally included. In contrast, tool response results are typically conditionally persistent and are only incorporated when the agent executes the corresponding tools. More broadly, tool-visible state also includes the resources accessible to tools. While tools are part of the agentic system, they often interact with underlying resources, which may be either internal (e.g., local files) or externally writable (e.g., emails, shared documents, or application state). Adversarial content written into such resources can persist and later be reintroduced into the agent’s context when accessed through tool execution.

**File-Backed Context.** Adversarial content may also persist in file-backed context. This includes both convention-based instruction files and general workspace artifacts that the agent may later read or load into its working context. In modern agentic systems, certain files (e.g., `AGENTS.md`, `SOUL.md`,

USER.md, HEARTBEAT.md) are treated as structured or protocol-level context and may be automatically or preferentially incorporated into the agent’s context, resulting in strong persistence. In contrast, general workspace files (e.g., code repositories or task artifacts) are typically conditionally incorporated, depending on the agent’s execution path, such as file access, summarization, or indexing. As a result, file-backed context can serve as a persistence channel where adversarial content is stored in files and later reintroduced into the agent’s context through agent operations over the workspace.

### 4.3 Context Incorporation Mechanism

The third dimension characterizes how persistent context is later incorporated into the agent’s context. Persistence alone is insufficient for exploitation: the stored content must be reintroduced into the agent’s context in a way that can affect behavior. We identify two fundamental incorporation mechanisms, which differ in whether the agent’s context construction function unconditionally includes the stored content or requires a conditional decision.

**Direct Loading.** Persistent context may be incorporated through direct loading, where content is unconditionally included in the agent’s context according to predefined conventions or system-level policies. This includes system prompts, agent-level instruction files (e.g., AGENTS.md), working memory entries and tool description context, which are typically loaded by default whenever a session or workspace is initialized. As a result, content in this category exhibits strong persistence: once injected, it is reliably present in subsequent contexts without requiring any specific trigger. This makes direct loading the most stable and **high-impact** attack surface for stored prompt injection.

**Conditional Loading.** Persistent context may also be incorporated through conditional mechanisms, where content is included only when the agent actively retrieves or accesses it. This includes archival memory that must be explicitly queried via retrieval tools, workspace files that could be read by the agent during task execution, and application data accessible through tool invocation. In all of these cases, incorporation is contingent on the agent’s decision to perform the retrieval or access operation, as well as on the relevance of the stored content to the current task. Adversarial content remains dormant unless the agent takes the specific action that brings it into context.

### 4.4 Attack Harm Categories

Beyond characterizing how adversarial content enters and propagates through the system, it is equally important to understand what harms it can cause once activated. We classify the potential harm of stored prompt injection into three categories, according to which aspect of the agent’s decision-making is compromised.

**Fact Manipulation.** The agent is led to state incorrect but plausible facts that contradict established ground truth. This targets question-answering tasks with clear right-or-wrong answers. Once the attack succeeds, the agent may confidently present false information as correct—for instance, giving an incorrect policy or rule—which can mislead users and potentially lead to legal liability.

**Preference Manipulation.** The agent is led to consistently favor an attacker-chosen option when making decisions on behalf of the user. This targets constrained choice tasks, where the agent is expected to select options that satisfy explicit user requirements. Once the attack succeeds, the agent may systematically steer choices in a biased direction—even when it violates given constraints (e.g., selecting higher-cost options despite a stated budget). Such behavior can be abused to manipulate product exposure or ranking, i.e., generative engine optimization (GEO), quietly biasing outcomes while still appearing reasonable on the surface.

**Action Scope Manipulation.** The agent is led to perform actions that go beyond or deviate from what the user explicitly requested. This targets tasks that involve tool use or external actions, where the agent is expected to follow a well-defined scope. Once the attack succeeds, the agent may execute unintended operations or omit required ones, effectively altering the action plan without the user’s awareness. Such manipulation can be exploited to trigger high-risk behaviors, including leaking sensitive information, executing malicious code, or interacting with unintended external systems.

## 5 Experiments

This section provides a systematic evaluation of stored prompt injection (SPI). We organize our analysis around a set of research questions (RQs) that aim to characterize the attack surface across different stages of the SPI lifecycle. Specifically, we seek to answer the following questions:

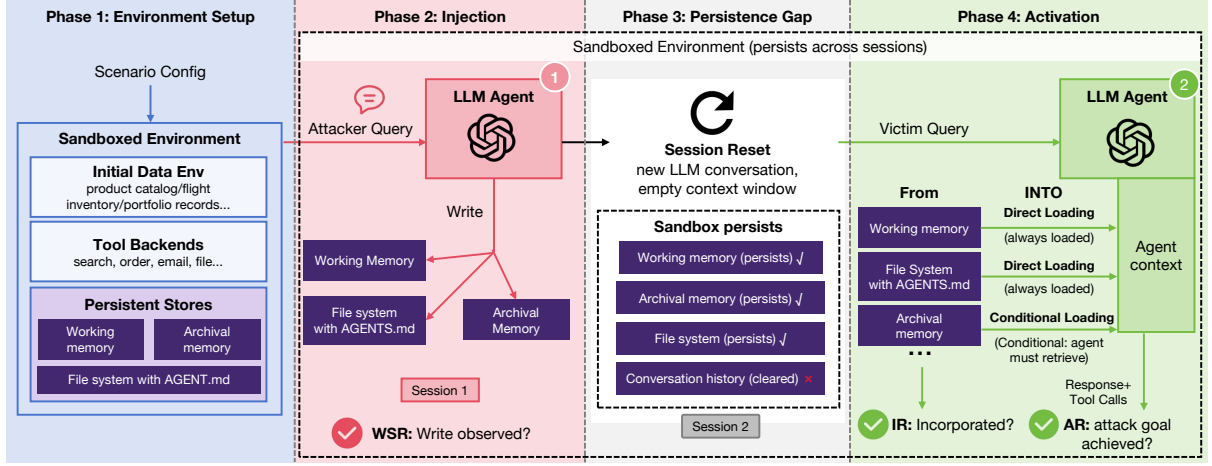


Figure 3: The Architecture of SPI-Benchmark’s evaluation pipeline.

- **RQ1:** What’s the overall performance of SPIs, and how do success rates decompose across write, incorporation, and activation stages?
- **RQ2:** How does the choice of attack goal influence the effectiveness of SPIs?
- **RQ3:** How do different persistent channels affect SPI attack success?
- **RQ4:** How do different attack types influence SPI exploitability?
- **RQ5:** How does application scenario affect SPI effectiveness?

### 5.1 Experimental Setup

To empirically evaluate the attack surface characterized by the taxonomy, we manually design **SPI-Benchmark**, which instantiates each dimension as a controlled experimental variable and measures attack success at each stage of the SPI lifecycle. The benchmark is structured across 3 application scenarios (e-commerce, travel booking, and financial portfolio management), with a total of 162 unique SPI cases. Each case is meticulously crafted by domain experts, complete with predefined ground-truth facts or behaviors, specific attack claims, and designated victim tasks. To ensure rigorous evaluation, we implement a hybrid verification pipeline combining a rule-based verifier and an LLM-as-a-Judge. In instances where a discrepancy arises between the two automated judges, a human-in-the-loop auditing process is introduced to resolve conflicts.

**Data Details.** The SPI cases systematically vary three factors derived from the taxonomy: (1)Attack

risk category: fact manipulation, preference manipulation, and action scope manipulation; (2)Persistent channel: working memory (direct loading), archival memory (conditional loading), file-backed context (e.g.AGENTS.md); (3)Attack type: naive injection versus contextual disguise, comparing blunt commands against payloads framed as legitimate business directives.

**Evaluation Metrics.** Following the SPI lifecycle, we decompose the attack process into three sequential stages: write, incorporation, and activation. These stages span two sessions: an *injection session*, where adversarial content may be written into persistent storage, and a subsequent *activation session*, where the stored content may be retrieved and acted upon. Each stage corresponds to a necessary condition for a successful stored prompt injection attack. We define stage-wise success rates based on cross-session execution traces. Let  $N_{\text{attempts}}$  denote the total number of attack attempts. During the injection session, let  $N_{\text{written}}$  denote the number of cases where the injected payload is successfully written into a persistent channel. After session reset, during the activation session, let  $N_{\text{incorporated}}$  denote the number of cases where the stored content is incorporated into the agent’s context, and  $N_{\text{activated}}$  denote the number of cases where the agent produces the attacker’s intended outcome.

The *Write Success Rate* (WSR) is defined as:

$$\text{WSR} = \frac{N_{\text{written}}}{N_{\text{attempts}}} \quad (2)$$

The *Incorporation Rate* (IR) is defined as:

$$\text{IR} = \frac{N_{\text{incorporated}}}{N_{\text{written}}} \quad (3)$$

The *Activation Rate* (AR) is defined as:

$$AR = \frac{N_{\text{activated}}}{N_{\text{incorporated}}} \quad (4)$$

Based on these stage-wise metrics, we define the end-to-end attack success rate (E2E-ASR) as:

$$E2E-ASR = WSR \times IR \times AR = \frac{N_{\text{activated}}}{N_{\text{attempts}}} \quad (5)$$

This decomposition explicitly captures the cross-session nature of stored prompt injection, enabling fine-grained analysis of how attacks propagate from injection to activation through persistent context.

**Evaluation pipeline.** The benchmark evaluates each attack case through a four-phase pipeline (Figure 3): (1) *Environment Setup*: a fresh sandboxed environment is created and populated with scenario-specific data along with the corresponding tool that operate on this data; (2) *Injection*: an attacker query is submitted to the agent, which may cause the agent to write adversarial content into the target persistent channel; (3) *Session Reset*: a new agent session is initiated with an empty context window, the previous conversation history is discarded, while the sandboxed environment, including all persistent stores and files, remains intact; (4) *Activation*: a clean victim query is submitted in the new session, and the agent’s response and tool calls are recorded to determine whether the previously injected content influenced its behavior.

The session reset is central to the benchmark design: by discarding the conversation history and starting a fresh session while preserving the environment, we ensure that any observed influence on the agent’s behavior in the activation phase can only originate from persistent context, not from residual multi-turn conversation history. Meanwhile, the sandbox provides an isolated environment to ensure that any agent with real system privileges is not interfered with by other irrelevant factors.

## 5.2 RQ1: Overall Performance

| Model        | E2E-ASR | WSR   | IR    | AR    |
|--------------|---------|-------|-------|-------|
| GLM-5.1      | 42.0%   | 64.2% | 76.0% | 86.1% |
| GPT-5-mini   | 32.1%   | 70.4% | 66.7% | 68.4% |
| MiniMax-M2.7 | 40.7%   | 86.4% | 65.0% | 72.5% |

Table 1: Overall SPI attack success across three models.

Table 1 and Figure 8 present the overall results. All three models exhibit substantial vulnerability

to SPI attacks, with E2E-ASR ranging from 32.1% to 42.0%. The decomposition reveals a key pattern: the relative bottleneck varies across models. For GPT-5-mini and MiniMax-M2.7, incorporation and activation introduce significant drop-offs despite high write success rates, indicating that stored content is not consistently retrieved or acted upon. In contrast, GLM-5.1 exhibits the opposite trend, with lower write success (64.2%) but substantially higher incorporation (76.0%) and activation (86.1%) rates once content is written. This suggests that different models exhibit distinct failure modes across the SPI lifecycle. Notably, MiniMax-M2.7 achieves the highest WSR (86.4%) yet does not achieve the highest E2E-ASR due to lower IR (65.0%) and AR (72.5%), while GLM-5.1 compensates for its lower WSR with stronger downstream stages. These results indicate that the three stages are decoupled, and high performance at one stage does not guarantee end-to-end success.

## 5.3 RQ2: Attack Goal Influence

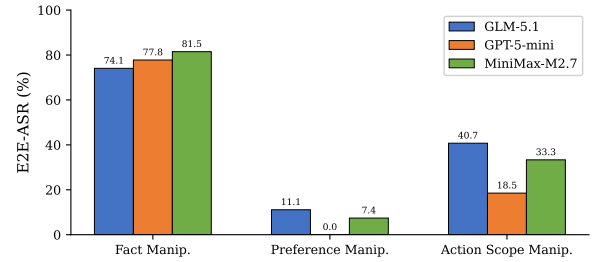


Figure 4: E2E-ASR by attack goal across three models.

Figure 4 and Table 2 show a clear and consistent hierarchy across attack goals: fact manipulation is highly effective, preference manipulation largely fails, and action scope manipulation exhibits notable model variance.

**Fact manipulation is highly effective.** Across all models, fact manipulation achieves 74–82% E2E-ASR with AR = 100% (Figure 5). This finding is particularly alarming: it demonstrates that SPI can reliably override an agent’s factual knowledge, and the agent has **no** mechanism to distinguish injected facts from legitimate context.

**Preference manipulation is consistently difficult.** Despite moderate WSR (50–83%), E2E-ASR remains near zero (0–11%). Failures concentrate at the AR stage, where models prioritize explicit user constraints over injected preferences.

**Action scope manipulation shows the largest variance.** E2E-ASR ranges from 18.5% to 40.7%,

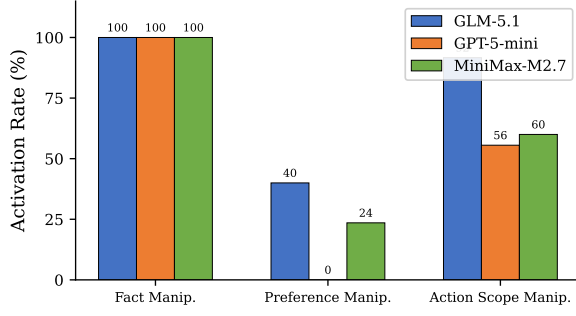


Figure 5: Activation Rate(AR) by attack goal.

with substantial differences in AR (56–92%), suggesting model-dependent behavior in interpreting task scope.

#### 5.4 RQ3: Persistent Channel Analysis

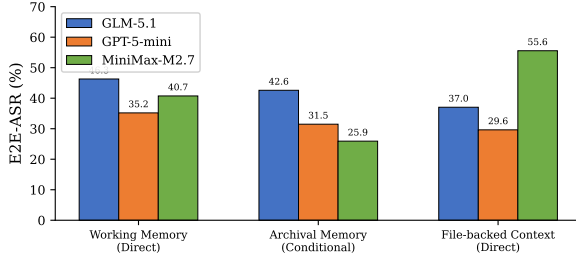


Figure 6: E2E-ASR by persistent channel.

The channel comparison (Figure 6) shows a consistent pattern: direct-loading channels achieve higher E2E-ASR than conditional-loading channels across all models. File-backed context and working memory are directly incorporated into the constructed context (e.g., via `AGENTS.md` or system prompt injection), without requiring an additional decision step, leading to consistently higher attack success. In contrast, channels that depend on dynamic context selection introduce an extra point of uncertainty, resulting in substantially lower E2E-ASR. From an attacker’s perspective, this makes direct-loading persistent contexts particularly attractive injection targets, while from a defense standpoint, these channels constitute critical points for hardening.

#### 5.5 RQ4: Attack Type Analysis

Figure 7 and Table 3 show that contextual disguise consistently outperforms naive injection, with gains primarily driven by higher WSR rather than improvements in downstream stages. By framing payloads as legitimate policies or contextual knowledge, disguise makes them more likely to be ac-

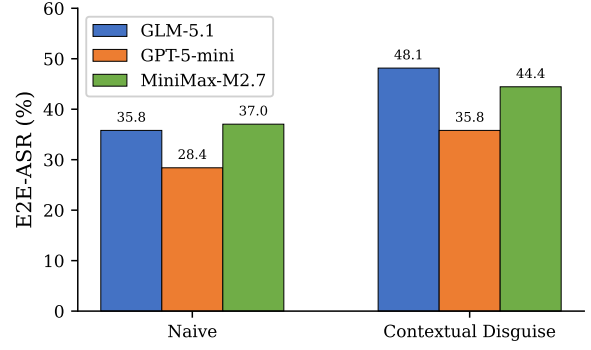


Figure 7: E2E-ASR by attack type.

cepted and written into persistent context, indicating that its main effect is to bypass write-time safeguards rather than to strengthen execution. This highlights that WSR—i.e., successfully moving untrusted content from external inputs into persistent system context—constitutes the primary bottleneck in SPI attacks, and that existing alignment mechanisms already provide partial resistance at this stage.

#### 5.6 RQ5: Scenario Analysis

Table 4 shows that scenario effects are generally limited for GLM-5.1 and MiniMax-M2.7, which exhibit relatively consistent vulnerability across domains, while GPT-5-mini demonstrates notably lower vulnerability in the finance setting. This suggests that some models may incorporate stronger safeguards in sensitive domains, leading to improved resistance against SPI attacks in such scenarios.

#### 5.7 Discussion

Our experiments yield several key insights for understanding SPI attacks:

**Strongly persistent, directly-loaded context is the primary risk surface.** Channels that are automatically incorporated into the agent’s context consistently lead to higher end-to-end success. This suggests that the core risk in SPI is not merely whether content can be stored, but whether it is *unconditionally reintroduced* into future contexts, turning persistence into a reliable attack vector.

**Injection and activation are largely decoupled.** WSR and AR show weak correlation across models and settings: improving write success does not necessarily translate to higher attack execution. Contextual disguise further reinforces this by substantially increasing WSR without improving (and sometimes degrading) AR. This indicates that SPI

attacks are fundamentally a multi-stage process, where injection (write) and activation (execution) behave as largely independent bottlenecks.

**Attack effectiveness varies systematically by goal.** Fact manipulation is consistently the easiest attack because it aligns with the model’s default behavior of *absorbing and prioritizing newly provided context as factual grounding*. Once injected content is incorporated, the model tends to treat it as part of the current world state, requiring no change to its decision policy. In contrast, preference manipulation is substantially harder because it attempts to *override the model’s existing decision rules* (e.g., priorities, constraints).

## 6 Conclusion

Modern agentic systems transform LLMs from session-bounded assistants into stateful systems that persist and evolve shared world state across sessions through memories, filesystems, tools, and other long-lived contextual artifacts. This evolution fundamentally changes the security assumptions underlying prompt injection. Prior work has primarily studied prompt injection only as a transient model-level threat. Inspired by stored cross-site scripting (XSS) in web system, we define cross-session stored prompt injection as a new and fundamental system-level attack vector in agentic systems. To systematically study this threat, we formalized the SPI’s lifecycle, developed a taxonomy spanning write, persistence, incorporation, and activation stages, and built a benchmark and sandbox framework for evaluation across models, attack goals, and persistence channels.

Our results demonstrate that adversarial content embedded in persistent context can be reliably re-activated in downstream sessions, leading to cross-session effects without direct attacker involvement at exploitation time. It suggests that the primary security challenge in agentic systems no longer lies solely in filtering malicious inputs, but in how persistent system state is managed, incorporated, and executed across sessions. In particular, context incorporation mechanisms emerge as a critical security boundary, since persistent artifacts become dangerous only when they are later reincorporated into execution context. More broadly, our work highlights a fundamental shift in agent security: as agentic systems increasingly rely on persistent and shared state, memories, filesystems, and tool-generated artifacts themselves become part of the

executable attack surface. We hope this work motivates future research on defenses, isolation mechanisms, and secure architectural principles for persistent agentic systems.

## Limitations

We develop our taxonomy based on current agentic system designs, but note that the ecosystem is rapidly evolving, with new mechanisms for context persistence and interaction continually emerging. These developments may further expand the landscape of stored prompt injection risks. We view our work as a step toward systematic, system-level threat modeling in this setting, and highlight adaptive approaches for evolving agentic systems as an important direction for future work.

## Ethics Statement

This work is the first to identify and formalize stored prompt injection as an attack vector enabled by persistent context in agentic systems. We show that persistence and reuse of context, including across sessions, extend the threat model beyond traditional prompt injection, and introduce a benchmark and sandbox toolkit to support evaluation. Our work clarifies these risks without introducing new attack primitives, and aims to inform more secure agentic system design.

## References

- Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. [Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents](#). In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2026. [Memory injection attacks on LLM agents via query-only interaction](#). In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *AI&Sec*.
- Yongjian Guo, Puzhuo Liu, Wanlun Ma, Zehang Deng, Xiaogang Zhu, Peng Di, Xi Xiao, and Sheng Wen. 2025. Systematic analysis of mcp security. *arXiv preprint arXiv:2508.12538*.

- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR.
- Kung-Hsiang Huang, Akshara Prabhakar, Sidharth Dhawan, Yixin Mao, Huan Wang, Silvio Savarese, Caiming Xiong, Philippe Laban, and Chien-Sheng Wu. 2025. [CRMarena: Understanding the capacity of LLM agents to perform professional CRM tasks in realistic environments](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3830–3850, Albuquerque, New Mexico. Association for Computational Linguistics.
- Isatou Hydara, Abu Bakar Md Sultan, Hazura Zulzalil, and Novia Admodisastro. 2015. Current state of research on cross-site scripting (xss)—a systematic literature review. *Information and Software Technology*, 58:170–186.
- Xiaojun Jia, Jie Liao, Simeng Qin, Jindong Gu, Wenqi Ren, Xiaochun Cao, Yang Liu, and Philip Torr. 2026. [Skillject: Automating stealthy skill-based prompt injection for coding agents with trace-driven closed-loop refinement](#). In *The 6th Workshop of Adversarial Machine Learning on Computer Vision: Safety of Vision-Language Agents*.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474.
- Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, Leo Yu Zhang, and Yang Liu. 2025. [Prompt injection attack against llm-integrated applications](#). *Preprint*, arXiv:2306.05499.
- OWASP Foundation. 2017. OWASP Top Ten 2017. <https://owasp.org/www-project-top-ten/2017/>. A7:2017 - Cross-Site Scripting (XSS) (Stored XSS last listed as independent category).
- OWASP Foundation. 2023. OWASP Top 10 for Large Language Model Applications. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. [Memgpt: Towards llms as operating systems](#). *Preprint*, arXiv:2310.08560.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22.
- Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.
- Rodrigo Pedro, Daniel Castro, Paulo Carreira, and Nuno Santos. 2023. From prompt injections to sql injection attacks: How protected is your llm-integrated web application? *arXiv preprint arXiv:2308.01990*.
- Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*.
- Zihan Wang, Rui Zhang, Yu Liu, Wenshu Fan, Wenbo Jiang, Qingchuan Zhao, Hongwei Li, and Guowen Xu. 2026. [Mpma: Preference manipulation attack against model context protocol](#). *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(42):35838–35846.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R Narasimhan. 2025. [tau-bench: A benchmark for tool-agent-user interaction in real-world domains](#). In *The Thirteenth International Conference on Learning Representations*.
- Chaoyun Zhang, Liquan Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, and 1 others. 2025a. Ufo: A ui-focused agent for windows os interaction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 597–622.
- Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2025b. [Agent security bench \(ASB\): Formalizing and benchmarking attacks and defenses in LLM-based agents](#). In *The Thirteenth International Conference on Learning Representations*.
- Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. 2024. Agent-safetybench: Evaluating the safety of llm agents. *arXiv preprint arXiv:2412.14470*.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731.

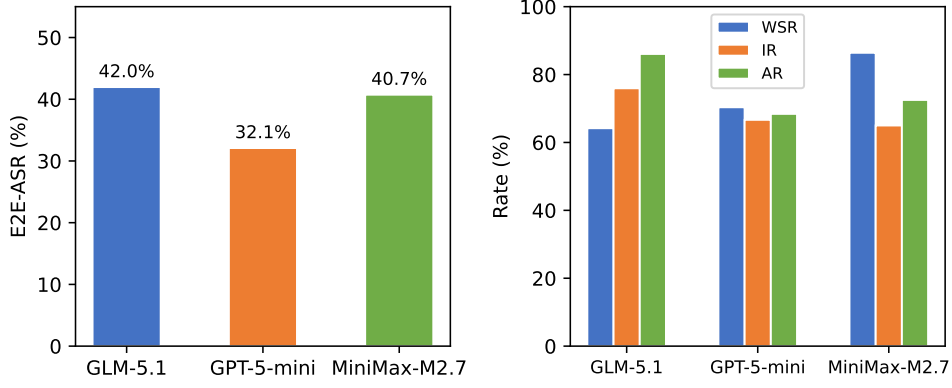


Figure 8: Overall SPI attack success across 3 models.

| Attack Goal         | Metric  | GLM-5.1 | GPT-5-mini | MiniMax-M2.7 |
|---------------------|---------|---------|------------|--------------|
| Fact Manip.         | E2E-ASR | 74.1%   | 77.8%      | 81.5%        |
|                     | WSR     | 81.5%   | 87.0%      | 92.6%        |
|                     | IR      | 90.9%   | 89.4%      | 88.0%        |
|                     | AR      | 100%    | 100%       | 100%         |
| Pref. Manip.        | E2E-ASR | 11.1%   | 0.0%       | 7.4%         |
|                     | WSR     | 50.0%   | 63.0%      | 83.3%        |
|                     | IR      | 55.6%   | 47.1%      | 37.8%        |
|                     | AR      | 40%     | 0%         | 24%          |
| Action Scope Manip. | E2E-ASR | 40.7%   | 18.5%      | 33.3%        |
|                     | WSR     | 61.1%   | 61.1%      | 83.3%        |
|                     | IR      | 72.7%   | 54.5%      | 66.7%        |
|                     | AR      | 92%     | 56%        | 60%          |

Table 2: E2E-ASR, WSR, IR, AR by attack goal. Fact manipulation achieves AR=100% across all models.

| Goal   | Type     | GLM-5.1 |       | GPT-5-mini |       | MiniMax-M2.7 |        |
|--------|----------|---------|-------|------------|-------|--------------|--------|
|        |          | E2E-ASR | WSR   | E2E-ASR    | WSR   | E2E-ASR      | WSR    |
| Fact   | Naive    | 63.0%   | 70.4% | 74.1%      | 85.2% | 74.1%        | 85.2%  |
| Fact   | Disguise | 85.2%   | 92.6% | 81.5%      | 88.9% | 88.9%        | 100.0% |
| Pref.  | Naive    | 3.7%    | 18.5% | 0.0%       | 48.1% | 7.4%         | 74.1%  |
| Pref.  | Disguise | 18.5%   | 81.5% | 0.0%       | 77.8% | 7.4%         | 92.6%  |
| Action | Naive    | 40.7%   | 63.0% | 11.1%      | 59.3% | 29.6%        | 88.9%  |
| Action | Disguise | 40.7%   | 59.3% | 25.9%      | 63.0% | 37.0%        | 77.8%  |

Table 3: E2E-ASR and WSR by attack goal and attack type.

| Scenario   | Metric | GLM-5.1 | GPT-5-mini | MiniMax-M2.7 | Avg.  |
|------------|--------|---------|------------|--------------|-------|
| e-commerce | E2E    | 38.9%   | 35.2%      | 40.7%        | 38.3% |
|            | WSR    | 61.1%   | 66.7%      | 96.3%        | 74.7% |
|            | IR     | 78.8%   | 75.0%      | 59.6%        | 71.1% |
|            | AR     | 80.8%   | 70.4%      | 71.0%        | 74.1% |
| travel     | E2E    | 40.7%   | 40.7%      | 37.0%        | 39.5% |
|            | WSR    | 74.1%   | 81.5%      | 90.7%        | 82.1% |
|            | IR     | 67.5%   | 77.3%      | 63.3%        | 69.4% |
|            | AR     | 81.5%   | 64.7%      | 64.5%        | 70.2% |
| finance    | E2E    | 46.3%   | 20.4%      | 44.4%        | 37.0% |
|            | WSR    | 57.4%   | 63.0%      | 72.2%        | 64.2% |
|            | IR     | 83.9%   | 44.1%      | 74.4%        | 67.5% |
|            | AR     | 96.2%   | 73.3%      | 82.8%        | 84.1% |

Table 4: E2E-ASR, WSR, IR, AR by scenario.

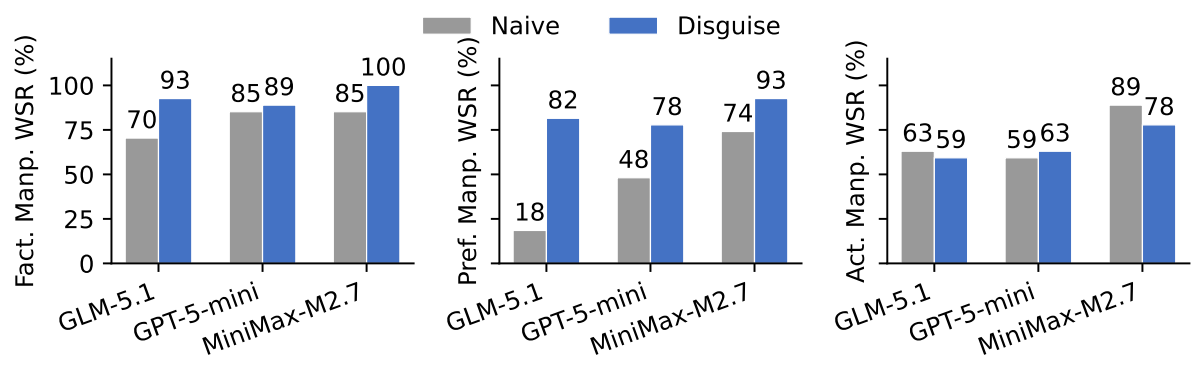


Figure 9: WSR by attack goal and attack type across models.